

High-Performance Numeric statistics

Prepared by: Team 4

- . Hamed Essam (BN: 6)**
- . Goda Saber (BN: 5)**
- . Mohamed Abdelmenoem (BN: 29)**

1. Introduction

In the era of Big Data, processing numerical information at scale is a critical challenge. When datasets reach the scale of hundreds of gigabytes, traditional single-threaded applications fail due to memory exhaustion and CPU bottlenecks. This report details the design and implementation of a **Distributed Numerical Statistics Framework**. The system leverages **MPI (Message Passing Interface)** for distributed computing and **Docker** for containerized orchestration, allowing for massive parallel processing of data across a cluster of virtualized nodes.

2. Project Objectives

- **Massive Parallelism:** Distributing the computational load across multiple CPUs/Nodes to reduce execution time.
 - **Memory Efficiency:** Implementing "Chunk-based processing" to handle 100GB+ of data on machines with limited RAM.
 - **Scalable Infrastructure:** Using Docker to allow the system to scale from 1 node to 100 nodes without changing the source code.
 - **Accurate Frequency Analysis:** Building a distributed histogram to identify data distribution patterns and top-frequent values.
-

3. System Architecture & Design

The framework is designed using a **Leader-Follower (Master-Worker) Paradigm**.

3.1. The Master Node (Rank 0)

The Master node acts as the orchestrator. Its responsibilities include:

- Parsing command-line arguments and calculating the global workload.
- Distributing specific data ranges to each Worker.
- Aggregating final results (Min, Max, Sum, Frequencies) and writing them to results.txt.

3.2. The Worker Nodes (Rank 1 to N)

Worker nodes are the "muscle" of the system:

- Generating local random data streams based on a unique seed.
- Performing local statistical calculations (Local Min, Local Max, Local Sum).
- Transmitting results back to the Master via MPI Reductions.

4. MPI Communication & Collective Operations

The Message Passing Interface (MPI) serves as the communication backbone of the system, enabling data exchange and synchronization between the isolated Docker containers. Rather than manually handling low-level socket programming, the framework utilizes MPI's robust abstraction layer to perform complex data reductions across the virtual network.

4.1. Collective Communication Patterns

To ensure the system remains efficient as it scales from 2 nodes to 100, we implemented **Collective Communication** instead of standard point-to-point sends. This allows the network hardware to optimize the path of data.

- **MPI_Reduce:** This is the primary operation used for statistical aggregation. It collects local results from every Worker and performs a mathematical operation (like sum or minimum) as the data moves toward the Master.
- **Rank-Based Logic:** The system uses `rank 0` as the dedicated collector. While all processes execute the same binary, conditional logic based on the process rank ensures that only the Master performs final I/O and reporting tasks.

4.2. Statistical Aggregation Types

We utilized specific MPI reduction types to handle different data requirements:

- **MPI_MIN & MPI_MAX:** These operations identify the global extreme values across the entire 10GB dataset by comparing the local extrema of every process.
- **MPI_SUM:** This aggregates the local sums into a global total. To handle the massive scale of 2.68 billion elements, we explicitly used the **MPI_LONG_LONG** data type to match our C-core's 64-bit summation variables, preventing data truncation or corruption during transfer.

4.3. Synchronization & Global State

The MPI environment provides a synchronized execution state.

- **Implicit Barriers:** Collective operations like `MPI_Reduce` act as synchronization points. The Master node will

automatically wait for the slowest Worker to finish its calculations before finalizing the report, ensuring that no data is lost due to network latency or CPU speed variances.

- **MPI_Abort:** In the event of a critical failure (such as an Out-Of-Memory error on a specific Worker), the system triggers a global abort. This prevents "zombie processes" where the rest of the cluster continues to run fruitlessly while waiting for a crashed node.

4.4. Scalability through Communicators

By using `MPI_COMM_WORLD`, the system treats the entire Docker cluster as a single computational unit. This design allows the user to change the number of processes in the `docker-compose` file or the `mpirun` command without modifying a single line of C code, providing true **horizontal scalability**.

```
void reduce_frequency(unsigned long long *local_freq,
                     unsigned long long *global_freq,
                     int rank)
{
    long long remaining = FREQ_SIZE;
    long long offset = 0;
    unsigned long long *tmp = NULL;
    long long batch;

    while (remaining > 0) {
        batch = (remaining < FREQ_BATCH) ? remaining : FREQ_BATCH;

        if (rank == 0) {
            tmp = (unsigned long long *)malloc(batch * sizeof(unsigned long long));
            if (!tmp) {
                fprintf(stderr, "OOM in reduce_frequency\n");
                MPI_Abort(MPI_COMM_WORLD, 1);
            }
        }

        MPI_Reduce(local_freq + offset,
                  (rank == 0) ? tmp : NULL,
                  (int)batch,
                  MPI_UNSIGNED_LONG_LONG,
                  MPI_SUM,
                  0,
                  MPI_COMM_WORLD);

        if (rank == 0) {
            memcpy(global_freq + offset, tmp, batch * sizeof(unsigned long long));
            free(tmp);
        }

        offset += batch;
        remaining -= batch;
    }
}
```

5. Global concatenation

```
/* ---- 4. Global reductions ---- */
int      global_min;
int      global_max;
long long global_sum;

MPI_Reduce(&local_min, &global_min, 1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD);
MPI_Reduce(&local_max, &global_max, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);
MPI_Reduce(&local_sum, &global_sum, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

unsigned long long *global_freq = NULL;
if (rank == 0) {
    global_freq = (unsigned long long *)calloc(FREQ_SIZE, sizeof(unsigned long long));
    if (!global_freq) {
        fprintf(stderr, "Cannot allocate global freq\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
}
reduce_frequency(local_freq, global_freq, rank);
free(local_freq);

double t_end = MPI_Wtime();
```

6. MPI Chunking

```
/* ---- 3. Process data in chunks ---- */
double t_start = MPI_Wtime();

long long remaining = local_count;
unsigned int seed = (unsigned int)(rank * 9973 + time(NULL));
srand(seed);

double local_data_gb = local_data_bytes / (1024.0 * 1024.0 * 1024.0);

while (remaining > 0) {
    long long this_chunk = (remaining < buf_size) ? remaining : buf_size;

    /* Generate random values in [VAL_MIN, VAL_MAX] */
    for (long long i = 0; i < this_chunk; i++) {
        buf[i] = (rand() % MAX_VAL) - HALF_MAX; /* maps [0, MAX_VAL) to [-HALF_MAX, HALF_MAX) */
    }

    for (long long i = 0; i < this_chunk; i++) {
        int v = buf[i];
        if (v < local_min) local_min = v;
        if (v > local_max) local_max = v;
        local_sum += v;
        local_freq[v + OFFSET]++; /* shift into [0, FREQ_SIZE) */
    }

    remaining -= this_chunk;
}
```

7. Challenges & Solutions

7.1. The Signed-to-Index Mapping Problem

The dataset contains signed integers: $[-500,000, +500,000]$. Since C arrays do not support negative indices, we implemented **Offset Mapping**.

- **Solution:** Apply a constant offset of $500,000$.
- **Formula:** $\text{Index} = \text{Value} + 500,000$.
- This maps the value $-500,000$ to index 0 and $+499,999$ to index $999,999$.

7.2. Overflow Safety for Large Sums

With a 10GB dataset containing billions of elements, a standard int would overflow during summation.

- **Solution:** The system uses long long (64-bit) for the running sum.
- **Capacity:** Max possible sum is $\approx 1.34 \times 10^{15}$, which is safely within the 9.22×10^{18} limit of a 64-bit signed integer.

7.3. Batched Frequency Reduction

Merging a 1-million-bucket frequency array (8MB) across 100 nodes can overwhelm network buffers.

- **Solution: Batched Reduction.** The array is split into four segments of 250,000 elements. The system reduces one segment at a time, clearing temporary memory between rounds.
-

8. Implementation Details: The C Core

- **Chunked Processing:** Instead of generating 10GB at once, the system uses a **Sliding Window**. It generates and processes data in 10-million-element chunks (~38MB). This ensures the memory footprint per process remains constant at ~46 MB regardless of the total data size.
 - **Workload Division:** To ensure a perfect split, the remainder of $(\text{Total} \div \text{Processes})$ is distributed one-by-one to the first few ranks.
-

9. Performance Analysis

Testing was conducted on a **10 GB dataset** (2,684,354,560 elements).

```
--- Data Info ---
Total elements      : 2684354560
Element size       : 4 bytes (int)
Total data size    : 10.00 GB (10737418240 bytes)
Value range       : [-500000, 500000]

--- Distribution Info ---
MPI processes      : 8
Elements per process : 335544320 (~1.25 GB)
Chunk size        : 10000000 elements (~38.15 MB)
Freq array per proc : 1000000 buckets (~7.63 MB)
Memory per process : ~45.78 MB (chunk + freq)

===== Results =====

--- Size Summary ---
Total data processed : 10.00 GB (2684354560 elements)
Data per process    : 1.25 GB (335544320 elements)
Memory used per proc : ~45.78 MB

--- Statistics ---
Min      : -500000
Max      : 499999
Sum      : -14817888531
Mean     : -55.200930

--- Performance ---
Total time      : 24.0967 seconds
Throughput     : 0.41 GB/s
Processes used  : 8
```

```
--- Data Info ---
Total elements      : 2684354560
Element size       : 4 bytes (int)
Total data size    : 10.00 GB (10737418240 bytes)
Value range       : [-500000, 500000]

--- Distribution Info ---
MPI processes      : 4
Elements per process : 671088640 (~2.50 GB)
Chunk size        : 10000000 elements (~38.15 MB)
Freq array per proc : 1000000 buckets (~7.63 MB)
Memory per process : ~45.78 MB (chunk + freq)

===== Results =====

--- Size Summary ---
Total data processed : 10.00 GB (2684354560 elements)
Data per process    : 2.50 GB (671088640 elements)
Memory used per proc : ~45.78 MB

--- Statistics ---
Min      : -500000
Max      : 499999
Sum      : -148194841774
Mean     : -52.226358

--- Performance ---
Total time      : 26.5316 seconds
Throughput     : 0.38 GB/s
Processes used  : 4
```


Observations:

1. **Scaling:** Moving from 2 to 4 processes nearly doubled the throughput, showing excellent horizontal scalability.
 2. **Mean Accuracy:** The calculated mean was near-zero (e.g., -0.003108), confirming the quality of the uniform random distribution.
-

10. Infrastructure: Docker & SSH (Deep Dive)

The infrastructure layer is the backbone that allows the C/MPI code to run across multiple "virtual machines" on a single host. By using Docker and SSH, we simulate a real-world Supercomputing Cluster.

10.2. Passwordless SSH: The Command & Control Bridge

MPI (specifically `mpirun`) requires a way to remotely start processes on other nodes. In a standard Linux environment, this would require a password every time a worker is accessed. To automate this for high-performance execution, we implemented a **Passwordless RSA Handshake**.

- **Key Generation at Build-Time:** During the `docker-compose build` phase, the Dockerfile executes `ssh-keygen` to create a public/private RSA key pair.
- **The Authorized Keys Mechanism:** The system is configured so that the **Public Key** is automatically added to the `authorized_keys` file of every container.
- **The SSH Daemon (sshd):** Each container runs an SSH server in the background. When you run the command on the Master, it uses the **Private Key** to "knock" on the Workers'

- doors. Since the Workers already recognize the Master's Public Key, they allow the Master to execute the computational task immediately without human intervention.
- **Strict Modes & Security:** The infrastructure setup includes specific commands to set file permissions (`chmod 600`) for the keys. MPI will refuse to run if it detects that the security keys are "too open" or accessible by other users.
-

10.3. The Hostfile: Defining the Topology

The final piece of the infrastructure is the **Hostfile**. This is a plain text file that acts as a map for the Master node.

Example Hostfile Structure:

Plaintext

```
mpi-master slots=4
mpi-worker1 slots=4
mpi-worker2 slots=4
```

- **Slots:** This tells the system how many CPU cores are available on each container. If we have 3 containers with 4 slots each, the infrastructure can handle a total of **12 parallel processes** (`-np 12`).
- **Orchestration:** When the user runs the system, the Master reads this file, connects via SSH to the hostnames listed, and launches the Worker processes simultaneously.

```

services:
  mpi-master:
    build: .
    container_name: mpi-master
    hostname: mpi-master
    networks:
      - mpi-net
    volumes:
      - shared-keys:/root/.ssh
      - ./output:/app/output
    entrypoint: >
    hash -c "
      /usr/sbin/sshd &&
      sleep 5 &&
      echo 'mpi-master slots=4' > /app/hostfile &&
      echo 'mpi-worker1 slots=4' >> /app/hostfile &&
      echo 'mpi-worker2 slots=4' >> /app/hostfile &&
      mpirun --hostfile /app/hostfile -np ${NP:-4} /app/stats ${ELEMENTS:-1000000}
    "

  mpi-worker1:
    build: .
    container_name: mpi-worker1
    hostname: mpi-worker1
    networks:
      - mpi-net
    volumes:
      - shared-keys:/root/.ssh

  mpi-worker2:
    build: .
    container_name: mpi-worker2
    hostname: mpi-worker2
    networks:
      - mpi-net
    volumes:
      - shared-keys:/root/.ssh

networks:
  mpi-net:
    driver: bridge

volumes:
  shared-keys:

```

11. Conclusion

The **Distributed Numerical Statistics Framework** successfully demonstrates that high-performance computing can be achieved on commodity hardware through effective parallelization and memory management. By combining MPI's low-level efficiency with Docker's portability, the system provides a robust solution for large-scale numerical analysis.